บทที่ 9

AI-Assisted Digital Circuit Design

ให้ AI ช่วยออกแบบวงจรดิจิทัลตลอดสาย

Spec → RTL → Testbench → Simulation → Hardware (DE0 / DE1) · worked example: Streaming Data Processing Pipeline

00 · AGENDA

ภาพรวมบทที่ 9

01

ทบทวน & คำถามนำ

เชื่อมจาก Counter → FSM → Testbench

02

เครื่องมือ AI 3 ระดับ

ผู้ช่วยสนทนา · โมเดลเฉพาะทาง · EDA Copilot

03

Workflow 6 ขั้นตอน

ให้ AI ช่วยตลอดสาย พร้อม human checkpoint

04

ตัวอย่างจริง: Data Pipeline

6 submodule · top-down spec · bottom-up test

05

จากซิมสู่บอร์ดจริง

Quartus → Pin Assignment → DE0 / DE1

06

ข้อจำกัดของ AI

Hallucination · ความเข้าใจ · IP/ความลับ

01 · ทบทวน

เชื่อมโยงกับบทก่อนหน้า

นักศึกษาผ่านอะไรมาแล้วก่อนถึงบทนี้

1. Fundamentals & Sequential Logic — module, wire/reg, flip-flop, counter (สัปดาห์ 3–6)
2. Moore FSM — Traffic Light Controller ออกแบบ state diagram เอง (สัปดาห์ 7–8)
3. Self-Checking Testbench — $display PASS/FAIL, $dumpvars, ModelSim (สัปดาห์ 9)

คำถามนำ: “ถ้าจะให้ AI ช่วยสร้าง Traffic Light Controller ตั้งแต่ต้นจนจบ AI ต้องรู้อะไรบ้าง และเราต้องตรวจอะไรบ้าง?”

ภูมิทัศน์เครื่องมือ AI สำหรับงานออกแบบดิจิทัล

จัดกลุ่มเป็น 3 ระดับ ตามความพร้อมใช้งานจริงในคาบ

ระดับ	ตัวอย่างเครื่องมือ	ใช้ทำอะไร	เหมาะกับ
1 · ผู้ช่วยสนทนาทั่วไป (ใช้จริงในคาบ)	Gemini, Claude, ChatGPT, GitHub Copilot / Cursor	สร้าง/อธิบาย/แก้ RTL, สร้าง testbench, อ่าน error	Workshop, การบ้าน, แล็บ
2 · โมเดล/เครื่องมือเฉพาะทาง (เชิงวิชาการ)	VeriGen, VerilogCoder (NVIDIA), RTLCoder, ChipNeMo, VerilogEval	งานวิจัย code generation เฉพาะ Verilog / วัดความสามารถ LLM	ตระหนักรู้ทันวงการ
3 · EDA AI Copilot เชิงพาณิชย์ (อุตสาหกรรม)	Synopsys.ai, Cadence Cerebrus + Verisium Debug, DeepMind AlphaChip	ช่วยทั้งสาย RTL-to-GDS: floorplan, timing, debug อัตโนมัติ	ปูพื้นเส้นทางอาชีพ

02 · เครื่องมือ AI

Prompt ที่ดี = สเปคที่ดี

หลัก prompt engineering สำหรับงานฮาร์ดแวร์

1. ระบุ input / output ให้ชัดเจน — ชื่อสัญญาณ, บิตวิดท์, ทิศทาง, active-high/low
2. แนบ truth table หรือ state diagram แทนคำอธิบายลอย ๆ
3. ระบุ timing requirement — synchronous/async, latency ที่ยอมรับได้
4. ระบุ edge case ที่ต้องจัดการ (เช่น backpressure, reset กลางคัน, ข้อมูลชนกัน)

02 · เครื่องมือ AI

เครื่องมือที่ใช้ลงมือทำจริงในแล็บ

จาก Tier 1 ในตารางก่อนหน้า — คอมโบที่ใช้ตลอดทั้ง workshop/แล็บของบทนี้

VS Code

Editor + Integrated Terminal ในตัวเดียว — เปิดไฟล์ RTL/TB, รันคำสั่ง compile/simulate, ดูผลลัพธ์ได้โดยไม่ต้องสลับโปรแกรม

GitHub Copilot

AI pair programmer — inline code suggestion ขณะพิมพ์ + Copilot Chat สำหรับสั่งงานด้วยภาษาธรรมชาติ (generate RTL/TB จาก spec, อธิบายโค้ด, แก้ error)

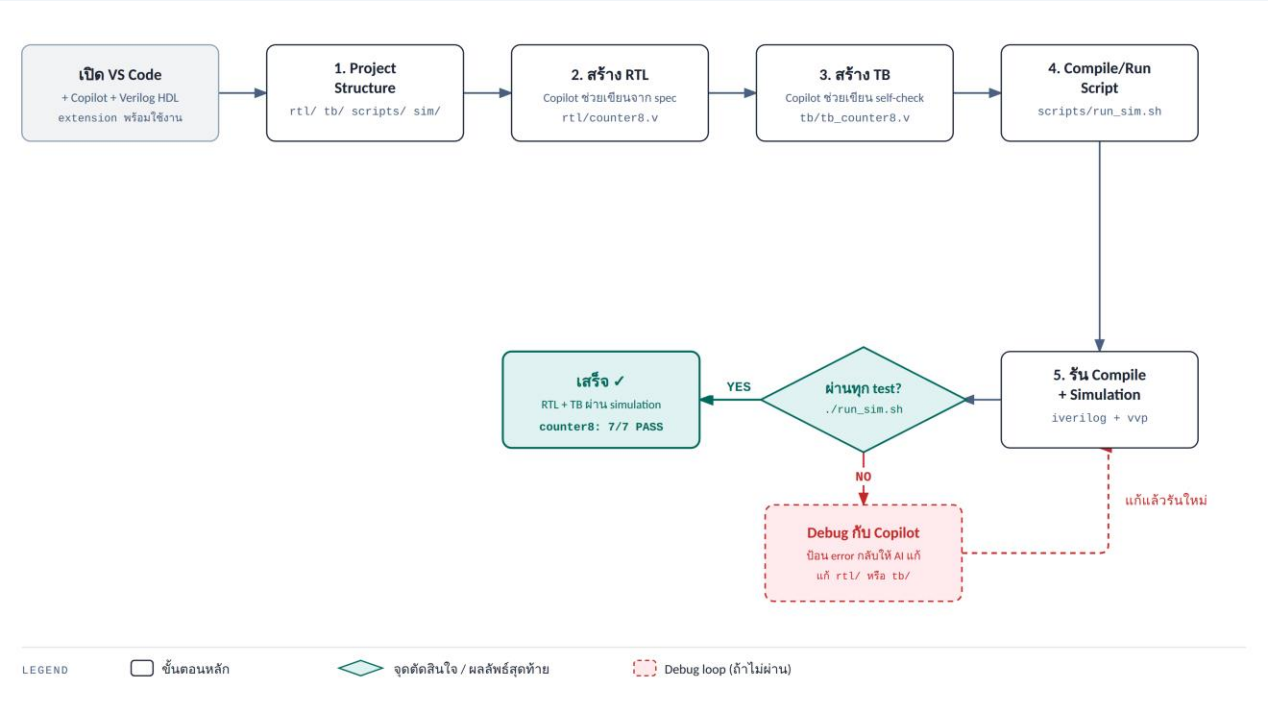
Verilog HDL Extension

Syntax highlighting, snippet, และตรวจ syntax เบื้องต้นสำหรับ .v/.sv ใน VS Code — ช่วยจับ typo ก่อนถึงขั้น compile จริงด้วยซ้ำ

สไลด์ถัดไป: workflow ที่ลงมือทำจริงด้วยเครื่องมือชุดนี้ ตั้งแต่โปรเจกต์ว่างเปล่าจนถึง RTL ที่ผ่าน simulation — สาธิตด้วยตัวอย่าง Counter 8-bit

Workflow ปฏิบัติจริง: Project ว่าง → RTL ผ่าน Sim

ตัวอย่าง: Counter 8-bit — ทุกไฟล์/คำสั่ง/ผลลัพธ์ในแผนภาพนี้รันจริงและผ่านจริงแล้ว



ตัวอย่าง Counter 8-bit (1/2): โครงสร้างโปรเจกต์ + RTL

ขั้น 1–2 ของ workflow — RTL ที่ Copilot ช่วยเขียนจาก spec สั้น ๆ: 8-bit up-counter, async reset, enable

โครงสร้างโปรเจกต์

```
counter8_demo/
├── rtl/
│   └── counter8.v
├── tb/
│   └── tb_counter8.v
├── scripts/
│   └── run_sim.sh
└── sim/
    └── (ผลลัพธ์การรัน)
```

rtl/counter8.v

```
module counter8 (
  input  wire       clk,
  input  wire       rst_n,
  input  wire       en,
  output reg  [7:0] count
);
  always @(posedge clk or
           negedge rst_n)
  begin
    if (!rst_n)
      count <= 8'd0;
    else if (en)
      count <= count+8'd1;
  end
endmodule
```

ตัวอย่าง Counter 8-bit (2/2): Testbench + ผลรัน Simulation จริง

ขั้น 3–5 ของ workflow — self-checking TB + script + ผลลัพธ์จริงจาก iverilog (ไม่ใช่ตัวอย่างสมมติ)

tb/tb_counter8.v (บางส่วน)

```
task check(
  input [191:0] name,
  input [7:0] expected);
begin
  if (count === expected)
    pass_count=pass_count+1;
  else
    fail_count=fail_count+1;
end
endtask

// เรียกใช้จริง:
en = 1;
repeat(5) @(negedge clk);
check("Test3-FiveIncs",8'd5);
```

$ ./scripts/run_sim.sh (ของจริง)

```
[PASS] Test1-AsyncReset
[PASS] Test2-HoldOnReset
[PASS] Test3-FiveIncrements
[PASS] Test4-HoldOnDisable
[PASS] Test5-BeforeWrap
[PASS] Test6-WrapTo0
[PASS] Test7-MidRunReset
======================
TOTAL: 7 PASS, 0 FAIL
RESULT: ALL TESTS PASSED
```

03 · WORKFLOW

Workflow หลัก: ให้ AI ช่วยตลอดสาย

6 ขั้นตอน พร้อม human checkpoint กำกับทุกขั้น

4.1	เขียน Spec/Requirement ให้ AI เข้าใจ — input/output, truth table, timing, edge case
4.2	Generate RTL Code จาก spec → checkpoint: อ่านโค้ดทุกบรรทัดเทียบกับสเปค
4.3	Code Review ร่วมกับ AI — หา logic error/hazard ที่มองข้าม
4.4	Generate Testbench ด้วย AI — self-checking $display PASS/FAIL → checkpoint: ครอบคลุม test case จริงหรือไม่
4.5	รัน Simulation จริง — รันจริง ไม่เชื่อ AI เฉย ๆ, ใช้ AI ช่วยอ่าน error/waveform
4.6	Debug Loop กับ AI — วนซ้ำจนผ่าน → checkpoint สุดท้ าย: เข้าใจว่าทำไมบั๊กเกิด ไม่ใช่แค่ AI แก้ให้ผ่าน

วงจรที่จะสาธิต: Streaming Data Processing Pipeline คืออะไร

บริบทการใช้งานจริงก่อนเข้าสู่โครงสร้างวงจร — จะสร้างทีละสเต็ปในสไลด์ถัดไป

สถานการณ์

ระบบ IoT/embedded ที่มีเซนเซอร์ (เช่น วัดอุณหภูมิ, แรงดัน, ความเร็วลม) ส่งค่าดิบ 8 บิตเข้ามาต่อเนื่องแบบสตรีม ทีละ 1 sample ต่อรอบ ไม่มีจุดสิ้นสุด

ปัญหาที่ต้องแก้

ค่าดิบมี noise/sensor glitch ปนมาเป็นบางครั้ง — ค่ากระโดดขึ้นลงจนดูแนวโน้มยาก — ไม่รู้ว่าค่าที่เคยเกิดสูงสุด/ต่ำสุดคือเท่าไร — ต้องการแจ้งเตือนอัตโนมัติเมื่อค่าผิดปกติ โดยไม่มีคนเฝ้าดูตลอดเวลา

หน้าที่ของวงจรนี้

รับ sample เข้ามาทีละค่า ประมวลผลแบบ real-time ให้ได้ผลลัพธ์ 4 อย่างพร้อมกันต่อ 1 sample ที่ผ่านเกณฑ์: ค่าเฉลี่ยเคลื่อนที่ (แนวโน้ม), ค่าต่ำสุด/สูงสุดสะสม (ประวัติ), และธง alarm (latch)

ต่อไปนี้จะสร้างวงจรนี้ทีละขั้น (bottom-up) ไม่ใช่โผล่มาสำเร็จรูปทีเดียว — เพื่อให้เห็นว่าทำไมแต่ละ stage ถึงต้องมีอยู่

โครงสร้างวงจร (Step 1/5): รับข้อมูลเข้า — FIFO Buffer

เริ่มสร้างทีละ stage (bottom-up) — นี่คือ stage แรกที่ต้องมี

FIFO Buffer

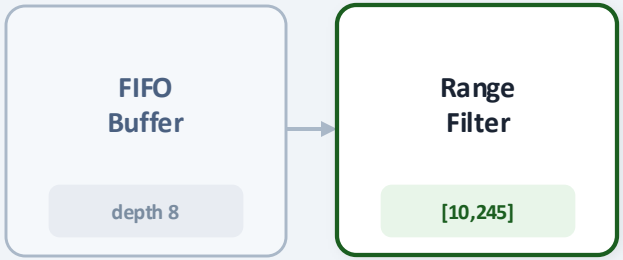
depth 8

ทำไมต้องมี stage นี้?

sensor กับวงจรประมวลผลอาจทำงานคนละจังหวะกัน (sensor ส่งข้อมูลไม่สม่ำเสมอ) — ถ้าไม่มีบัฟเฟอร์กันไว้ก่อน ข้อมูลอาจหายไปตอนที่วงจรประมวลผลยังไม่พร้อมรับ FIFO Buffer (depth 8) จึงต้องเป็น stage แรกเสมอ เพื่อรองรับความเร็วที่ไม่ตรงกันนี้ — ตอนนี้ยังไม่มีการกรองหรือคำนวณอะไร ทั้งนั้น แค่รับเข้ามาเก็บไว้ก่อนตามลำดับเดิม (FIFO order)

โครงสร้างวงจร (Step 2/5): กรองข้อมูลขยะทิ้ง — Range Filter

ต่อจาก FIFO Buffer — stage ที่ 2

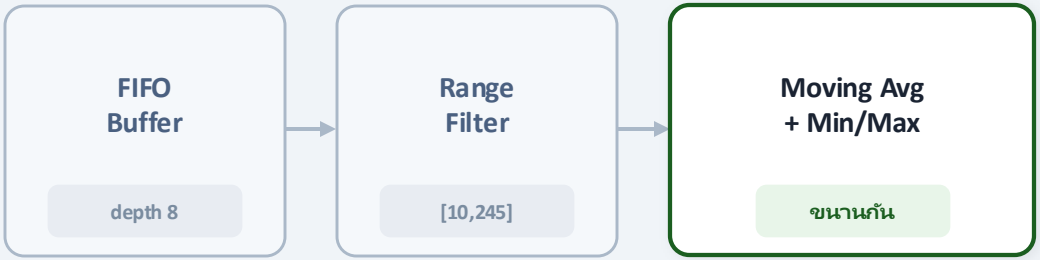


ทำไมต้องมี stage นี้?

เซนเซอร์บางครั้งส่งค่าผิดปกติสุดขั้ว (glitch) เช่น ต่ำกว่า 10 หรือสูงกว่า 245 ซึ่งไม่ใช่ค่าจริงจากการวัด ถ้าปล่อยให้ค่าพวกนี้หลุดเข้าไปคำนวณต่อ ค่าเฉลี่ย/min/max จะผิดเพี้ยนไปหมด — Range Filter จึงต้องมาเป็น stage ที่สอง คัดค่านอกช่วง [10,245] ทิ้งก่อน ไม่ให้หลุดผ่านไปยัง stage ถัดไปเลย

โครงสร้างวงจร (Step 3/5): แนวโน้ม — Moving Avg + Min/Max

แตกออกเป็น 2 สายขนานกันเป็นครั้งแรก — stage ที่ 3

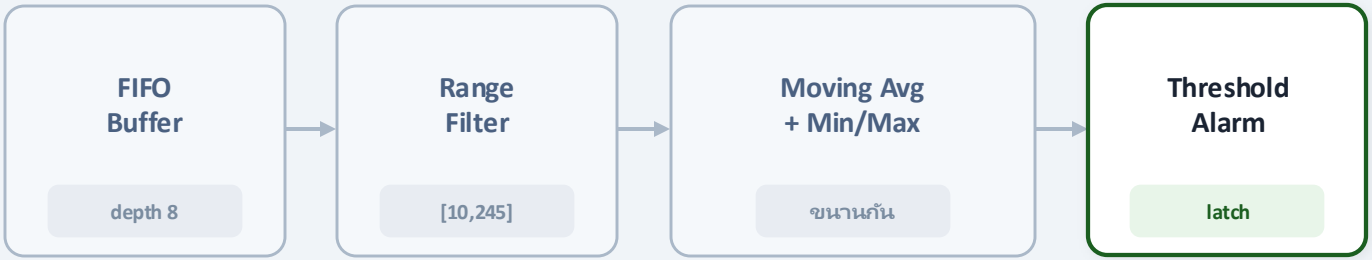


ทำไมต้องมี stage นี้?

ข้อมูลที่ผ่านการกรองแล้วยังกระโดดขึ้นลงตามธรรมชาติของเซนเซอร์ ดูแนวโน้มยากถ้าดูค่าดิบทีละจุด — Moving Average (เฉลี่ย 4 samples ล่าสุด) ช่วยสรุปแนวโน้มให้เรียบขึ้น ส่วน Min/Max Tracker เก็บค่าต่ำสุด/สูงสุดที่เคยเกิดสะสมไว้ ทั้งสองโมดูลนี้กินข้อมูลจากจุดเดียวกัน (Range Filter) และทำงานขนานกันอิสระต่อกัน

โครงสร้างวงจร (Step 4/5): จับความผิดปกติ — Threshold Alarm

ต่อจาก Moving Average — stage ที่ 4



ทำไมต้องมี stage นี้?

มีค่าเฉลี่ยแล้ว แต่ยังต้องมีคนเฝ้าดูตลอดเวลาว่าค่าเกินขีดอันตรายหรือยัง — Threshold Alarm เปรียบเทียบค่าเฉลี่ยกับ threshold (200) อัตโนมัติทุกครั้งที่มีค่าใหม่ และต้อง "จำ" ไว้ว่าเคยเกินแล้ว (latch ค้าง) จนกว่าจะมีคนสั่งล้าง (clear) — ไม่ใช่แค่แจ้งเตือนแวบเดียวแล้วหายไปเอง

โครงสร้างวงจร (Step 5/5): รวมผลเป็นแพ็กเก็ต — ครบสมบูรณ์

6 submodule ต่อกันเป็นสาย — เน้น spec แบบ Top-Down + build/ทดสอบแบบ Bottom-Up



Output Packetizer รวมผลจากทุกโมดูล (avg, min, max, alarm) ให้ออกมาเป็น 1 แพ็กเก็ตต่อ 1 sample ที่ผ่าน filter พร้อมกันในจังหวะเดียว — ครบทั้ง 6 submodule แล้ว

Top-Down Spec (01_spec.md)

เขียนสเปคระดับบนสุดก่อน (interface + กติกาทั้งระบบ) แล้วค่อยแตกลงเป็น 6 submodule พร้อม interface ของแต่ละตัว

Bottom-Up Build & Test

สร้าง + unit test ทีละโมดูลจากซ้ายไปขวา (FIFO ก่อน) ก่อนค่อยรวมเป็น integration test สุดท้าย — ต่างจาก Data Router รอบก่อนที่โมดูลเดียวจบ

4.1 สเปคระบบ (Top-Down): Input / Output ของ Data Pipeline

อินเทอร์เฟซระดับบนสุดที่ AI ต้องรู้ก่อนเริ่มแตกเป็น submodule

สัญญาณ	ทิศทาง	ความหมาย
clk, rst_n	in	นาฬิกา, รีเซ็ต active-low
in_data[7:0]	in	ค่าตัวอย่าง (sample) ที่จะป้อนเข้าระบบ
in_valid	in	in_data ใช้งานได้ในไซเคิลนี้
in_ready	out	ระบบพร้อมรับ sample ใหม่ (มาจาก FIFO ไม่เต็ม)
out_valid	out	มีแพ็กเก็ตผลลัพธ์พร้อมใช้งาน (1 ไซเคิล pulse)
out_avg[7:0]	out	ค่าเฉลี่ยเคลื่อนที่ของ 4 samples ล่าสุดที่ผ่าน filter
out_min[7:0] / out_max[7:0]	out	ค่าต่ำสุด/สูงสุดสะสมตั้งแต่ reset
out_alarm	out	สถานะ alarm (latch ค้างจนกว่าจะ clear)
alarm_clear	in	สัญญาณล้าง alarm latch

กติกาสำคัญจากสเปค: (1) ค่านอกช่วง [10,245] คือขยะ ต้องกรองทิ้งก่อนคำนวณใดๆ (2) avg คิดจาก ≤4 samples ที่ผ่าน filter เท่านั้น ห้ามหารด้วย 4 คงที่ตอน warm-up (3) alarm latch ค้างจนกว่าจะ clear (4) min/max สะสม ไม่ reset เอง

ตัวอย่างการคำนวณจริง: ป้อนสตรีมข้อมูลผ่านทั้งระบบ

สตรีม 10 ค่า (มีค่าขยะปน 2 ค่า) — ตัวเลขตรงกับผล integration test 9/9 PASS จริง

#	in_data	ผ่าน Range Filter?	avg	min	max	alarm
1	5	✗ ทิ้ง (< 10)	—	—	—	—
2	40	✓ ผ่าน	40	40	40	0
3	60	✓ ผ่าน	50	40	60	0
4	250	✗ ทิ้ง (> 245)	—	—	—	—
5	80	✓ ผ่าน	60	40	80	0
6	100	✓ ผ่าน	70	40	100	0
7	220	✓ ผ่าน	115	40	220	0
8	220	✓ ผ่าน	155	40	220	0
9	220	✓ ผ่าน	190	40	220	0
10	220	✓ ผ่าน	220	40	220	1 ←

แถวที่ 10: avg=220 > threshold(200) เป็นครั้งแรก → alarm เด้งจาก 0 เป็น 1 และค้างอยู่แบบนี้ (latch) แม้ avg รอบถัดไปจะลดลงก็ตาม จนกว่าจะมี alarm_clear

Submodule 1/6 — fifo_buffer.v: บัฟเฟอร์กันความเร็วไม่ตรงกัน

Synchronous FIFO, depth 8 — push เข้า / pop ออก เรียงตามลำดับเดิม (FIFO order)

สัญญาณ	I/O	ความหมาย
wr_valid	in	ขอเขียนค่าใหม่เข้า FIFO
wr_data[7:0]	in	ค่าที่จะเขียน
full	out	FIFO เต็ม (depth 8) ห้ามเขียนเพิ่ม
rd_ready	in	ขอ pop ค่าออก
rd_valid	out	rd_data ใช้งานได้ในไซเคิลนี้
rd_data[7:0]	out	ค่าที่ pop ออกมา
empty	out	FIFO ว่าง ห้าม pop

พฤติกรรม (Behavior)

push เมื่อ wr_valid && !full → เขียนที่ wr_ptr แล้วเลื่อนตัวชี้ / pop เมื่อ rd_ready && !empty → อ่านที่ rd_ptr แล้วเลื่อนตัวชี้ (rd_data ออกช้า 1 ไซเคิล) / wr_ptr กับ rd_ptr มีบิตเกิน (AW+1) เพื่อแยกแยะ full กับ empty เวลาตัวชี้วนมาตรงกันพอดี

ตัวอย่างการคำนวณ (Example)

push 40 → 60 → 80 ติดกัน 3 ไซเคิล (wr_valid=1) แล้วเริ่ม pop รัวๆ (rd_ready=1) → ได้ค่าออกมาตามลำดับเดิม 40, 60, 80 (First-In-First-Out) — ถ้า pop ตอน FIFO ว่าง (empty=1) rd_valid จะเป็น 0 เสมอ ไม่ pop ค่าเก่าซ้ำ

Submodule 2/6 — range_filter.v: กรองค่านอกช่วงทิ้ง

ช่วงที่ยอมรับ [LO=10, HI=245] — ค่านอกช่วงถือเป็น sensor glitch

สัญญาณ	I/O	ความหมาย
in_valid	in	in_data ใช้งานได้ในไซเคิลนี้
in_data[7:0]	in	ค่าตัวอย่างดิบจาก FIFO
out_valid	out	out_data ผ่านเกณฑ์ใช้ได้ (registered)
out_data[7:0]	out	ค่าที่ผ่านเกณฑ์เท่านั้น

พฤติกรรม (Behavior)

out_valid <= in_valid && (in_data ≥ 10 && in_data ≤ 245) — เช็คช่วงแบบ combinational (in_range) แล้ว register ผลทุกไซเคิล จึงมาช้ากว่า in_valid 1 ไซเคิลเสมอ — โมดูลถัดไปต้องเช็ค out_valid ก่อนใช้ out_data ทุกครั้ง

ตัวอย่างการคำนวณ (Example)

```
in_data=5   → 5<10 นอกช่วง        → out_valid=0 (ทิ้ง)
in_data=40  → อยู่ในช่วง [10,245] → out_valid=1 (ผ่าน)
in_data=250 → 250>245 นอกช่วง     → out_valid=0 (ทิ้ง)
```

Submodule 3/6 — moving_avg.v: ค่าเฉลี่ยเคลื่อนที่ 4 samples

Sliding window ขนาด 4 — จัดการช่วง warm-up ก่อนข้อมูลครบ 4 ค่า

สัญญาณ	I/O	ความหมาย
in_valid	in	sample ที่ผ่าน filter แล้ว ใช้งานได้
in_data[7:0]	in	ค่า sample ที่ผ่าน filter
avg_valid	out	avg_data ใช้งานได้ในไซเคิลนี้
avg_data[7:0]	out	ค่าเฉลี่ยของ ≤4 samples ล่าสุด

พฤติกรรม (Behavior)

เก็บ 4 samples ล่าสุดใน window[0..3] แบบวนซ้ำ (circular), sum อัปเดตแบบ sliding (บวกค่าใหม่ ลบค่าเก่าที่หลุดหน้าต่าง) — fill_count นับจำนวน sample จริงที่เคยเห็น (อิ่มตัวที่ 4) เลือกตัวหารที่ถูกต้อง (1,2,3,4) ระหว่าง warm-up แทนหารด้วย 4 คงที่

ตัวอย่างการคำนวณ (Example)

ป้อน 40 → 60 → 80 → 100 → 220 ตามลำดับ:
avg = 40, 50, 60, 70, 115

คำนวณตัวสุดท้าย: (60+80+100+220)/4 = 115
(40 หลุดหน้าต่างออกไปแล้ว)

Submodule 4/6 — minmax_tracker.v: ค่าต่ำสุด/สูงสุดสะสม

ค่าต่อเนื่อง (continuous) ตั้งแต่ reset — ไม่มี valid flag เพราะไม่ reset เองระหว่างทาง

สัญญาณ	I/O	ความหมาย
in_valid	in	sample ที่ผ่าน filter แล้ว ใช้งานได้
in_data[7:0]	in	ค่า sample ที่ผ่าน filter
min_data[7:0]	out	ค่าต่ำสุดสะสมตั้งแต่ reset
max_data[7:0]	out	ค่าสูงสุดสะสมตั้งแต่ reset

พฤติกรรม (Behavior)

reset ตั้ง min=0xFF(255) และ max=0x00 เพื่อให้ sample แรกที่เข้ามาชนะเสมอ — ทุกครั้งที่ in_valid เทียบ sample ใหม่กับค่าปัจจุบัน ถ้าต่ำ/สูงกว่าค่อยอัปเดต — เป็นค่าต่อเนื่อง ไม่มี valid flag เพราะไม่ reset เองระหว่างทาง

ตัวอย่างการคำนวณ (Example)

ป้อน 40 → 60 → 80 → 100 → 220 ตามลำดับ:
min = 40 คงที่ตลอด (เป็นค่าต่ำสุดตั้งแต่ตัวแรก)
max = 40 → 60 → 80 → 100 → 220 (ไล่ตามค่าที่มากขึ้น)

Submodule 5/6 — threshold_alarm.v: แจ้งเตือนเกิน Threshold

Threshold = 200 — alarm ต้อง latch ค้างจนกว่าจะมี clear เท่านั้น

สัญญาณ	I/O	ความหมาย
avg_valid	in	avg_data ใช้งานได้ในไซเคิลนี้
avg_data[7:0]	in	ค่าเฉลี่ยจาก moving_avg
clear	in	สั่งล้าง alarm latch
alarm	out	สถานะ alarm (registered, latch)
alarm_live	out	= ผลปัจจุบัน OR alarm (combinational)

พฤติกรรม (Behavior)

ตั้ง alarm<=1 เมื่อ avg_valid && avg_data>200 เท่านั้น — ไม่มี else จึงค้างค่าเดิมทุกไซเคิลที่ไม่เข้าเงื่อนไข set/clear (latch จริง) — alarm_live เป็นสัญญาณเสริม combinational ให้โมดูลพี่น้องเห็นผล sample ปัจจุบันทันที ไม่ต้องรอ 1 ไซเคิลเหมือน alarm

ตัวอย่างการคำนวณ (Example)

avg มาเป็น 40→50→…→190 → alarm=0 ตลอด
avg=220 (>200) → alarm เด้งเป็น 1 และค้างอยู่แบบนี้ แม้ avg รอบถัดไปจะลดลงต่ำกว่า 200 ก็ตาม จนกว่าจะมี clear=1

Submodule 6/6 — output_packetizer.v: รวมผลเป็นแพ็กเก็ตเดียว

sync กับจังหวะ avg_valid — รวมผลจากทุกโมดูลเป็น 1 แพ็กเก็ตต่อ 1 sample

สัญญาณ	I/O	ความหมาย
avg_valid	in	จังหวะหลักของทั้งแพ็กเก็ต
avg_data[7:0]	in	ค่าเฉลี่ยจาก moving_avg
min_data / max_data	in	ค่าต่ำสุด/สูงสุดจาก minmax_tracker
alarm_in	in	ต่อกับ alarm_live (ไม่ใช่ alarm)
out_valid	out	มีแพ็กเก็ตผลลัพธ์พร้อมใช้งาน
out_avg/min/max/alarm	out	ค่าทั้งหมด register พร้อมกัน

พฤติกรรม (Behavior)

out_valid <= avg_valid — ค่าที่เหลือทั้งหมด register ตามไปพร้อมกันทุกไซเคิลที่ avg_valid=1 — ใช้ alarm_live (ไม่ใช่ alarm ตรง ๆ) เป็น alarm_in เพื่อแก้บั๊ก pipeline alignment (Bug 3/3): ถ้าใช้ alarm ตรง ๆ แพ็กเก็ตจะสะท้อนผลของ sample ก่อนหน้า

ตัวอย่างการคำนวณ (Example)

avg_valid pulse มาพร้อม avg=220, min=40, max=220, alarm_in=1
→ ไซเคิลถัดไป: out_valid=1, out_avg=220, out_min=40, out_max=220, out_alarm=1 (1 แพ็กเก็ตต่อ 1 sample ที่ผ่าน filter)

4.3 Code Review ร่วมกับ AI — ทีละ Submodule

รีวิวเทียบสเปคก่อน unit test — ตั้งข้อสงสัยล่วงหน้าได้ 2 จุด

Submodule	ผลรีวิว	ข้อสังเกต/ความเสี่ยง
fifo_buffer.v	ผ่าน	full/empty logic มาตรฐาน
range_filter.v	ผ่าน	boundary ครบทั้งสองด้าน
moving_avg.v	⚠ สงสัย	หารด้วย 4 คงที่ทุกครั้ง ไม่มี logic แยกช่วง warm-up ตามสเปค
minmax_tracker.v	ผ่าน	ค่าเริ่มต้น min/max ถูกต้อง
threshold_alarm.v	⚠ สงสัย	ดูเหมือน “คำนวณใหม่ทุกครั้ง” ไม่ใช่ latch ตามสเปค
output_packetizer.v	ผ่าน	timing 1-cycle ตรงกับโมดูลอื่น

ทั้งสองจุดจะถูกยืนยันด้วย unit test เดี่ยวของโมดูลนั้นโดยตรง (ไม่ต้องรอ integration เหมือน Data Router)

4.4 – 4.5 Unit Test ทีละโมดูล (Bottom-Up)

รันจริงด้วย iverilog ทีละไฟล์ — ก่อนรวมระบบ

1	`tb_fifo_buffer.v`	✓ 8/8 PASS
2	`tb_range_filter.v`	✓ 7/7 PASS
3	`tb_moving_avg.v`	✗ 1/4 PASS (3 FAIL)
4	`tb_minmax_tracker.v`	✓ 5/5 PASS
5	`tb_threshold_alarm.v`	✗ 2/4 PASS (2 FAIL)
6	`tb_output_packetizer.v`	✓ 3/3 PASS

2 จาก 6 โมดูล FAIL ตั้งแต่ระดับ unit test — ตรงกับที่ code review ตั้งข้อสงสัยไว้ล่วงหน้าทั้งสองจุด (ดูสไลด์ถัดไป)

4.2 Top Module: data_pipeline_top.v — เชื่อม 6 Submodule

ขั้นตอนสุดท้ายของ bottom-up build (ลำดับที่ 7) — ต่อสายตาม data path เดียวกับสเปค 4.1

พอร์ตระดับบนสุด (ตรงกับสเปค 4.1)

```
module data_pipeline_top (
  input  wire       clk,
  input  wire       rst_n,
  input  wire       in_valid,
  input  wire [7:0] in_data,
  output wire       in_ready,
  input  wire       alarm_clear,
  output wire       out_valid,
  output wire [7:0] out_avg,
  output wire [7:0] out_min,
  output wire [7:0] out_max,
  output wire       out_alarm
);
```

เชื่อมสาย 5 สเตจ (excerpt จริงจากไฟล์ .v)

```
assign in_ready = !fifo_full;

range_filter u_filter(
  .in_valid(fifo_rd_valid),
  .in_data (fifo_rd_data));

moving_avg u_avg(                  // ขนาน
  .in_valid(filt_valid));
minmax_tracker u_minmax(           // กับ
  .in_valid(filt_valid));          // บรรทัดบน

output_packetizer u_pkt(
  .alarm_in(alarm_live));  // ไม่ใช่ alarm!
```

โครงสร้างระดับบนสุด: 5 สเตจต่อเรียงกัน + แตกขนานที่ stage 3 (moving_avg ‖ minmax_tracker) แล้วรวมกลับที่ output_packetizer — in_ready = !fifo_full คือสัญญาณ backpressure เดียวของทั้งระบบ

4.5–4.6 ทดสอบ Top Module (Integration Test) (1/3)

tb_data_pipeline_top.v — ป้อนสตรีมจริงผ่านทั้งระบบ เทียบกับ reference model

เงื่อนไข	รายละเอียด
Stimulus	Phase 1: สตรีม 18 samples ปนขยะ 3 กลุ่มติดกันรวด (3, 2, 4 ค่า) — ทดสอบว่า filter กรองขยะที่ติดกันหลายค่าได้ถูกต้อง ไม่กระทบ warm-up/window
Reference model	testbench คำนวณ avg/min/max/alarm เองแบบ software สำหรับทุก sample ที่ผ่าน filter (ไม่พึ่ง DUT) — เดินตาม event list เดียวกับ driver เป๊ะ
Background capture	ดัก output packet ตามลำดับจริงที่ออกบน out_valid — order-based check (pattern เดียวกับ Data Router)
เงื่อนไข PASS	จำนวนแพ็กเก็ตของ Phase 1 = 9 และทุกฟิลด์ตรงกับ reference model 100% — ต่อด้วย alarm_clear (2/3) และ reset กลาง

transcript จริง (iverilog/vvp — รันจริง ไม่ใช่ตัวอย่างสมมติ)

```
--- Phase 1: สตรีมยาวขึ้น + ขยะติดกันหลายค่ารวด ---
[PASS] pkt0 avg=0x28 alarm=0   ...   [PASS] pkt6 avg=0xbe alarm=0
[PASS] pkt7 avg=0xdc alarm=1   [PASS] pkt8 avg=0xca alarm=1
(ต่อ Phase 2: alarm_clear ในสไลด์ถัดไป)
```

Integration Test (2/3): alarm_clear หลัง Latch

pulse alarm_clear หลัง latch แล้วป้อนค่าสูงกลับมา — ยืนยัน re-trigger ได้จริง + latch ไม่ auto-clear (สเปคข้อ 3)

เงื่อนไข	รายละเอียด
Stimulus	เข้า Phase นี้ alarm latch ค้างที่ 1 จาก Phase 1 (avg สุดท้าย=202>200) → pulse alarm_clear 1 ไซเคิล → ป้อนค่าต่ำ (50) → ป้อนค่าสูงสุด (245) ติดกัน 4 ค่ารวด → ป้อนค่าต่ำ (50) อีกครั้ง
Reference model	ขยายจาก Phase 1 ด้วย event ชนิดใหม่ EV_CLEAR (ล้าง alarm_latched=0 ทันทีตอนประมวลผล) — ไม่ต้องแก้ logic sliding-window เดิมเลย
เงื่อนไข PASS	(a) หลัง clear ต้องเห็น alarm=0 ทันที (b) เมื่อ avg ทะลุ threshold อีกครั้งต้อง re-trigger alarm=1 ได้จริง (c) alarm latch ค้าง

transcript จริง (iverilog/vvp — รันจริง ไม่ใช่ตัวอย่างสมมติ)

```
--- Phase 2: alarm_clear หลัง latch ---
[PASS] pkt9  avg=0xa0 alarm=0   <- เพิ่ง clear สำเร็จ
[PASS] pkt10 avg=0xa6 ... pkt12 avg=0xc4  alarm=0
[PASS] pkt13 avg=0xf5 alarm=1   <- re-trigger กลับมาสำเร็จ
[PASS] pkt14 avg=0xc4 alarm=1   <- latch ค้างต่อ ไม่ auto-clear
```

Integration Test (3/3): Reset กลางสตรีม

assert rst_n=0 ระหว่างมีข้อมูลไหลผ่านทั้งสาย (mid-stream reset) — ตรวจว่า min/max/avg/alarm ถูกล้างจริง

เงื่อนไข	รายละเอียด
Stimulus	ป้อน valid 3 ค่า (60,90,120) ให้ min/max/alarm มีสถานะสะสมชัดเจนก่อน → รอ pipeline ระบายจนหมด → assert rst_n=0 ค้าง 4 ไซเคิลกลางสตรีม → ปล่อย rst_n → ป้อนสตรีมใหม่ 4 ค่า (100,110,120,130)
Reference model	ขยายด้วย event ชนิดใหม่ EV_RESET (ล้าง sum/fill/min/max/alarm กลับค่าเริ่มต้น mirror ฮาร์ดแวร์ async reset เป๊ะ) — filt_count (index) ไม่ reset เพื่อให้เช็คแพ็กเก็ตต่อเนื่องได้
เงื่อนไข PASS	แพ็กเก็ตแรกหลัง reset ต้องเห็น min=max=ค่าตัวเองล้วน ๆ (ไม่ติดค้างจาก min=40/max=245 ก่อนหน้า) และ alarm=0 (ไม่ latch ค้างจาก Phase 2) — พิสูจน์ reset ล้างทุก submodule จริง ไม่ใช่แค่ unit test เดี่ยว

transcript จริง (iverilog/vvp — รันจริง ไม่ใช่ตัวอย่างสมมติ)

```
--- Phase 3: reset กลางสตรีม (mid-stream rst_n) ---
[PASS] pkt15-17 (ก่อน reset) min=0x28 max=0xf5 alarm=1
[PASS] pkt18 (หลัง reset)  avg=0x64 min=0x64 max=0x64 alarm=0
          <- min/max/alarm ล้างจริง ไม่ติดค้างจากก่อน reset
[PASS] pkt19-21 avg ไต่ปกติ 0x69,0x6e,0x73  min=0x64 คงที่
```

Bug 1/3 (unit-level): moving_avg.v — Warm-Up Period

พบจาก tb_moving_avg.v โดยตรง ไม่ต้องรอ integration

transcript จริง (unit test)

```
[FAIL] warm-up 1/4: sum=40
  avg=0x0a=10 (expected 40)
[FAIL] warm-up 2/4: sum=80
  avg=0x14=20 (expected 40)
[FAIL] warm-up 3/4: sum=120
  avg=0x1e=30 (expected 40)
[PASS] sample 4/4: avg=40
```

root cause: หารด้วย 4 คงที่ (>>2) ทุกครั้ง ทั้งที่ window ยังไม่เต็ม 4 samples จริง — ค่าศูนย์ที่ยังไม่ถูกเขียนทับถูกรวมด้วย ทำให้ค่าเฉลี่ยต่ำเกินจริง

แก้ไข (fix)

```
// เดิม
avg_data <= sum >> 2;

// แก้แล้ว: หารด้วย
// จำนวนจริง (fill_count)
case (fill_count)
 0: avg<=new_sum;
 1: avg<=new_sum>>1;
 2: avg<=new_sum/3;
 default:
   avg<=new_sum>>2;
endcase
```

Bug 2/3 (unit-level): threshold_alarm.v — ไม่ Latch จริง

พบจาก tb_threshold_alarm.v โดยตรง — ตรงกับที่ code review ตั้งข้อสงสัยไว้

transcript จริง (unit test)

```
[PASS] step1: avg=50 -> alarm=0
[PASS] step2: avg=210 -> alarm=1
[FAIL] step3: avg drops to 50
  alarm should STAY latched=1
  got alarm=0 (expected 1)
[FAIL] step4: after clear=1
  alarm=0 (false-positive PASS)
```

root cause: alarm <= (avg_data > THRESH); คำนวณใหม่ทุกครั้ง ไม่ใช่ latch — พอค่าเฉลี่ยลดกลับต่ำกว่า threshold, alarm auto-clear เองทันที ทั้งที่สเปคห้าม

แก้ไข (fix)

```
// เดิม (ไม่ latch)
end else if (avg_valid)
  alarm <=
   (avg_data>THRESH);

// แก้แล้ว (latch จริง)
end else if (avg_valid
  && (avg_data>THRESH))
  alarm <= 1'b1;
// ไม่มี else -> คงค่าเดิม
```

Bug 3/3 (integration-only): Pipeline Alignment

ทุก unit test ผ่านครบแล้ว — แต่ integration test ยังเจอ bug ใหม่!

transcript จริง (integration test)

```
packet 7 : got avg=0xdc alarm=0
expected avg=0xdc alarm=1
```

`avg=220 (>threshold 200)` แต่ `out_alarm` ในแพ็กเก็ตเดียวกันยังเป็น `0`

root cause: threshold_alarm กับ output_packetizer ถูก trigger ด้วย avg_valid สัญญาณเดียวกัน แต่ output_packetizer อ่านค่า alarm (registered output) ที่ยังไม่ทันอัปเดตของไซเคิลนี้ — ได้ค่าของ sample ก่อนหน้าไปแทน

ทำไม unit test ไม่เจอ?

tb_threshold_alarm.v ทดสอบแค่ port “alarm” เดี่ยว ๆ ถูกต้องอยู่แล้ว — บั๊กนี้เกิดจาก “การต่อสายระหว่างโมดูล” ไม่ใช่ตรรกะภายในโมดูลไหนเลย จึงเห็นได้เฉพาะตอนรวมระบบเท่านั้น

แก้ไข: เพิ่มพอร์ต alarm_live

```
assign alarm_live = alarm |
  (avg_valid && avg_data>THRESH);
```

แล้วป้อน `alarm_live` (ไม่ใช่ `alarm`) เข้า `output_packetizer` แทน — ให้เห็นผลของ `sample` ปัจจุบันทันที

ผลลัพธ์สุดท้าย: Unit 31/31 + Integration 9/9 PASS

รันซ้ำหลังแก้ทั้ง 3 บั๊ก — ทุก assertion ผ่านครบ

✓ unit test ทั้ง 6 โมดูล จับบั๊กได้ 2 จุดตั้งแต่ระดับโมดูลเดียว (moving_avg, threshold_alarm) — isolate เร็ว ไม่ต้องไล่ข้าม 6 โมดูลพร้อมกัน

✓ integration test ยังจำเป็น! จับบั๊กที่ 3 ซึ่งเกิดจาก “การต่อสาย” ระหว่างโมดูล — ไม่มีในโมดูลไหนเลย เจอได้เฉพาะตอนรวมระบบ

✓ code review ทำนายบั๊กที่ 1–2 ได้ถูกทั้งคู่ล่วงหน้า — ยืนยันคุณค่าของการอ่านโค้ดเทียบสเปคก่อนรัน simulation เสมอ

✓ เทียบ Data Router (โมดูลเดียว): bottom-up ช่วยจำกัดขอบเขตการ debug ได้แคบกว่า แต่ยัง “ไม่ทดแทน” integration test ได้ทั้งหมด

ผ่าน Simulation ครบ 9/9

ไม่ได้แปลว่าใช้งานบนฮาร์ดแวร์จริงได้ทันที

ขั้นต่อไปของ AI-assisted workflow: นำ RTL ที่ผ่านการ verify แล้ว ไป synthesize ด้วย Quartus II, กำหนด pin assignment เข้ากับสวิตช์/ปุ่ม/จอ 7-segment บนบอร์ด Altera DE0 / DE1 จริง แล้ว program ผ่าน USB-Blaster เพื่อตรวจสอบพฤติกรรมบนฮาร์ดแวร์จริง

05 · ฮาร์ดแวร์

ขั้นตอน Synthesis ด้วย Quartus II

แปลง RTL ที่ผ่าน simulation แล้ว ให้กลายเป็นวงจรจริงบน FPGA

1. สร้าง Quartus Project ใหม่ เลือก device ให้ตรงกับบอร์ด (Cyclone III สำหรับ DE0 / Cyclone II สำหรับ DE1)

2. เพิ่มไฟล์ RTL ทั้ง 7 ไฟล์ที่ผ่าน unit + integration test ครบแล้ว (data_pipeline_top.v + submodule 6 ไฟล์) เข้าโปรเจกต์ — ไม่ใช้ไฟล์ testbench

3. กำหนด Top-Level Entity และ compile ครั้งแรกเพื่อดู syntax/elaboration error ระดับ synthesis

4. ใช้ AI ช่วยอ่าน Fitter/Compilation Report เบื้องต้น ก่อนไปกำหนด pin assignment

Pin Assignment: ผูกสัญญาณเข้ากับบอร์ดจริง

ตัวอย่างการ mapping สำหรับสาธิต Data Pipeline บน DE0 / DE1

สัญญาณในดีไซน์	อุปกรณ์บนบอร์ด (DE0/DE1)	หน้าที่
rst_n	KEY[0] (active-low)	รีเซ็ตวงจรทั้งระบบ
clk (ผ่าน clock divider)	สัญญาณนาฬิกาออนบอร์ด 50 MHz	หารความถี่ลงให้ debounce ปุ่ม inject ได้เสถียร
in_data[7:0]	SW[7:0]	ตั้งค่า sample ที่จะป้อนเข้าก่อนกดปุ่ม inject
in_valid	KEY[1] (กด = inject 1 sample)	ป้อนค่าจาก SW เข้าสตรีมทีละ 1 sample ต่อการกด
alarm_clear	KEY[2]	ล้าง alarm latch หลังตรวจสอบแล้ว
out_avg[7:0]	HEX1 – HEX0	แสดงค่าเฉลี่ยเคลื่อนที่ปัจจุบันเป็นเลขฐาน 16
out_valid	LEDG0	ไฟติด = มีแพ็กเก็ตผลลัพธ์ใหม่พร้อมใช้งาน
out_alarm	LEDR0	ไฟแดงติด = alarm latch (เคยเกิน threshold)

หมายเหตุ: out_min/out_max ต่อเพิ่มเข้า HEX2–HEX5 ได้แบบเดียวกันถ้าบอร์ดมีจอพอ · รายวิชานี้ใช้ DE0 เป็นหลัก หลักการเดียวกันใช้กับ DE1 ได้ เพียงปรับ pin ตาม .qsf จริง

ปรับ RTL สำหรับฮาร์ดแวร์จริง

สิ่งที่ testbench ไม่ต้องกังวล แต่บอร์ดจริงต้องมี

Simulation-only Testbench

- clk จาก always #5 clk=~clk; ในโค้ด — ไม่มีจริงบนบอร์ด
- reset ควบคุมด้วยโค้ด ตรงเวลาที่ต้องการเป๊ะ
- input ขับตรงด้วย in_data0=…; สมบูรณ์แบบ ไม่มี bounce
- รันเร็วเท่าที่ CPU จำลองได้ ดูผลได้เป็นหน่วย ns

Board-Ready RTL (DE0/DE1)

- ต้องเพิ่มโมดูล Clock Divider หาร 50 MHz ลงหลายพันเท่า ให้ตาเห็น LED/HEX เปลี่ยน
- รีเซ็ตผ่านปุ่มกด KEY จริง มักต้องผ่าน synchronizer
- input จากสวิตช์ทางกลมี contact bounce (~ms) ต้อง debounce หรือ sample ให้ช้าพอ
- ความเร็วจริงจำกัดด้วย critical path — ดูจาก Timing Analyzer หลัง compile

05 · ฮาร์ดแวร์

Compile → Program → ตรวจสอบบนฮาร์ดแวร์จริง

ขั้นตอนสุดท้ายของ workflow AI-assisted — จากไฟล์ .v สู่ไฟบนบอร์ดจริง

1. Compile เต็มรูปแบบใน Quartus II — ได้ไฟล์ .sof (SRAM Object File) สำหรับโปรแกรมชั่วคราว
2. ต่อบอร์ด DE0/DE1 ผ่านสาย USB-Blaster เปิด Programmer เลือกไฟล์ .sof และโหมด JTAG
3. กด Program แล้วรอจน Progress ถึง 100% — วงจรเริ่มทำงานจริงบน FPGA ทันที
4. ทดสอบจริง: ตั้งค่า SW[7:0] แล้วกด KEY[1] เพื่อ inject sample ทีละค่า ดู HEX/LEDG/LEDR ว่าตรงกับ reference model ที่ integration testbench เคย verify ไว้หรือไม่

ฮาร์ดแวร์เห็นอะไรที่ Simulation มองไม่เห็น

Simulation คือขั้นจำเป็น แต่ไม่ใช่ขั้นสุดท้ายของการ verify

ประเด็น	Simulation (Icarus/ModelSim)	ฮาร์ดแวร์จริง (DE0/DE1)
Timing	อุดมคติ delay=0 เว้นแต่ระบุเอง	propagation delay จริง ต้องผ่าน Timing Analyzer (slack) ก่อน
สัญญาณอินพุต	ขับด้วยโค้ด testbench สะอาด	สวิตช์/ปุ่มจริงมี contact bounce ต้อง debounce
ความเร็ว clock	จำลองกี่ MHz ก็ได้ในพริบตา	จำกัดตาม speed grade ของ FPGA และ I/O standard
การสังเกตผล	$display/$monitor ดูค่าได้ทุก signal ภายใน	เห็นได้เฉพาะ pin ที่ route ออก LED/HEX เท่านั้น
บั๊กที่เจอ	logic bug, pipeline alignment (เช่น Data Pipeline)	timing violation, glitch, metastability

06 · ข้อจำกัด

ข้อจำกัดและความเสี่ยงของ AI — Critical Thinking

ย้ำว่า AI เป็นผู้ช่วย ไม่ใช่ผู้ตัดสินใจสุดท้าย

01

Hallucination

AI สร้าง syntax/logic ที่ดูสมเหตุสมผลแต่ผิดได้ — ต้อง simulate/verify เสมอ ห้ามเชื่อโดยไม่ตรวจ

02

ไม่รู้ constraint จริง

AI ไม่รู้ timing, area, power ของฮาร์ดแวร์จริง ถ้าไม่ระบุใน prompt ให้ชัดเจน

03

ต้องอธิบายได้เอง

ผู้ออกแบบต้องอธิบายโค้ดที่ AI สร้างได้เอง โดยเฉพาะบริบทสอบ/ประเมินผล

04

ความเป็นส่วนตัว / IP

ข้อควรระวังเมื่อใช้เครื่องมือ AI แบบคลาวด์กับโค้ดที่เป็นความลับในบริบทอุตสาหกรรม

07 · WORKSHOP

Workshop สั้นในคาบ (~30 นาที)

ลองทำ mini-workflow เองเป็นคู่

1. จับคู่ เลือกวงจรง่าย 1 วงจร (เช่น 2-to-4 Decoder หรือ FSM ใหม่ที่ไม่ซ้ำของเดิม)
2. ทำ workflow ข้อ 4 ทั้งหมดด้วยตนเอง โดยใช้ AI ช่วยทุกขั้นตอน: spec → RTL → review → testbench → simulate → debug
3. บันทึกจุดที่ AI ตอบผิด/ต้องแก้เอง อย่างน้อย 1 จุด พร้อมอธิบายว่าทำไมถึงผิด
4. เปรียบเทียบผลลัพธ์/เวลาที่ใช้ กับตอนเขียนเองด้วยมือในบทก่อน ๆ — อภิปรายสั้น ๆ ท้ายคาบ

สรุปบทที่ 9

AI ช่วยได้ตลอดสาย spec → RTL → testbench → simulation → debug — แต่มนุษย์ต้องตรวจทุกขั้น (human checkpoint) ✓

worked example Data Pipeline (6 submodule): top-down spec → bottom-up unit test เจอบั๊กจริง 3 จุด (2 ระดับโมดูล + 1 ระดับ integration) แก้จริงจนผ่านครบ ✓

simulation ผ่านครบ ไม่ได้แปลว่าใช้งานบนฮาร์ดแวร์ได้ทันที — ต้องผ่าน synthesis, pin assignment, timing analysis บน DE0/DE1 ✓

AI มีข้อจำกัด: hallucination, ไม่รู้ constraint จริง, ต้องอธิบายโค้ดได้เอง, ความเป็นส่วนตัว/IP ✓

▶ สัปดาห์ถัดไป: ALU & RISC-V Single Cycle Pipeline (W10) → Memory Design (W11) → Timing Analysis (W12) → Basic SoC (W13–14)